⚛ React Learning — Deep Session Notes

From an Angular Developer's Perspective

> **Audience:** Beginner in React | Strong Angular background
> **Goal:** Self-study, course reference, and first project starter guide
> **Mentor style:** Sharp comparisons, practical depth, no fluff

---

📖 Table of Contents

1. What is a SPA (Single Page Application)?
2. Why React? Advantages & Disadvantages
3. DOM vs Virtual DOM vs Incremental DOM
4. JSX — JavaScript XML
5. Components — The Building Blocks
6. Props & Prop Drilling
7. Hooks — Superpowers for Functions
8. Angular vs React — Full Comparison Map
9. React & Node.js — The Relationship
10. Creating a React Project on Linux (VS Code + Vite)
11. App.jsx — The Root Component
12. Fragment — The Invisible Wrapper
13. Folder Structure
14. export default — Making Components Public
15. CSS in React — 3 Ways to Style
16. Bootstrap in React
17. Tailwind CSS in React
18. React Execution Flow — From URL to DOM
19. Component Invocation — `<Header />` vs `{Header()}`
20. Quick Reference — Traps & Memory Hooks
21. 🗺 Master Summary Flow

---

1. What is a SPA (Single Page Application)?

Core Idea

A SPA loads **one single HTML file once**, then uses JavaScript to dynamically swap out views as the user navigates — without ever triggering a full browser reload.

Think of it like a restaurant: you sit at one table (one HTML page), and the kitchen keeps bringing you different courses (different views). You never have to leave the restaurant and come back.

The Problem It Solves

Traditional (Multi-Page) websites required the server to send a brand-new HTML page for **every single link click**. This caused:

- Slow, blank-screen reloads
- Heavy server load on every navigation
- Poor, clunky user experience

Before → After

	Multi-Page App (MPA)	Single Page App (SPA)
How it works	Server renders & sends a new HTML page per route	Server sends one `index.html`; JS handles everything else
Navigation	Full page reload with blank screen	Instant in-memory component swap
Data fetching	HTML rendered server-side	Only raw JSON fetched from APIs
UX	Clunky, noticeable loading	Smooth, app-like feel

How React Implements SPA Routing

React is **not** a full framework, so routing is not built-in. You install `react-router-dom`:

```
npm install react-router-dom
```

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

export default function App() {
  return (
    <BrowserRouter>
      <nav>
        {/* Link intercepts clicks — prevents browser reload */}
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>

      <Routes>
        {/* Swaps components based on the current URL */}
        <Route path="/"       element={<Home />} />
        <Route path="/about"  element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

How `react-router-dom` Stops the Reload

It intercepts the click event, **prevents the default browser behavior**, and uses the browser's native **HTML5 History API** (`pushState`) to change the URL visually — while React swaps components in memory. No reload ever happens.

Angular vs React — Routing

	Angular	React
Routing	Built-in `@angular/router` (included by default)	Must install `react-router-dom` separately
Configuration	Highly structured, enforced via `RouterModule`	Flexible, component-based

> ⚠ **Trap:** Using a standard `<a href="/about">` instead of `<Link to="/about">`. The `<a>` tag triggers a full-page reload and **destroys your SPA state**.

> **Memory Hook:** SPA = One HTML shell, infinite JavaScript views.

---

2. Why React? Advantages & Disadvantages

Core Idea

React is a **declarative UI library** — you describe *what* the UI should look like based on your data (state), and React figures out *how* to update the screen efficiently.

Advantages

Advantage	What It Means in Practice
⚡ Lightning-fast updates	Virtual DOM diffs only what changed — no wasted DOM writes
🧩 Reusable components	Build once, use anywhere (e.g., a `<Button />` used on 50 pages)
🌍 Massive ecosystem	Libraries for everything: routing, state, animations, testing
🔄 Predictable data flow	Data flows in one direction (parent → child), easy to trace bugs
🧑‍🤝‍🧑 Huge community	Abundant tutorials, packages, job market demand

Disadvantages

Disadvantage	What It Means in Practice
⚠ Unnecessary re-renders	Without optimization (`React.memo`), components re-render too often
⚠ You assemble the stack	No built-in router, HTTP client, or form system — you choose libraries
⚠ JSX learning curve	Writing HTML inside JavaScript feels weird at first
⚠ Decision fatigue	Too many options for state management (Redux, Zustand, Context API...)

> 🧠 **Memory Hook:** React gives you the engine. You build the car.

---

3. DOM vs Virtual DOM vs Incremental DOM

What is the Real DOM?

The **Document Object Model (DOM)** is the browser's giant tree structure representing every element on a webpage. Updating it is expensive — changing one node can trigger reflows and repaints across the entire tree.

Analogy: Rewriting an entire book page just to fix one typo.

React's Solution: Virtual DOM

The Virtual DOM is a **lightweight JavaScript object** (a copy of the real DOM tree) kept in memory.

How React uses it:

```
State changes → React re-runs your component function
             → Builds a NEW Virtual DOM tree
             → Compares it to the OLD Virtual DOM (DIFFING)
             → Finds exactly what changed
             → Updates ONLY those nodes in the Real DOM (RECONCILIATION)
```

Analogy: Photocopying two versions of a book and using "Find & Replace" to change only the typo.

```
export default function Counter() {
  const [count, setCount] = useState(0);

  // Calling setCount triggers the Virtual DOM diffing process
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

Angular's Solution: Incremental DOM

Angular does **not** use a Virtual DOM. Instead:

1. During build time, Angular **compiles** your HTML templates into step-by-step JavaScript instructions.
2. When data changes, it runs those instructions — which point **directly** to the exact Real DOM node — and updates it in-place.
3. No second copy of the UI tree is ever kept in RAM.

Analogy: Having a GPS coordinate map. Angular flips directly to the right page and corrects the typo with a pen.

Practical Example: Changing a Footer Year

React's approach:

```
export default function Layout() {
  const [year, setYear] = useState(2024);

  // When year changes → React re-runs Layout() entirely
  // → Builds new Virtual DOM for <main> AND <footer>
  // → Diffs both → Finds only {year} changed → Updates that text node only
  return (
    <div>
      <main>Lots of heavy content here...</main>
      <footer onClick={() => setYear(2025)}>
        Copyright {year}
      </footer>
    </div>
  );
}
```

Angular's approach (conceptually):

```
Build time: Compiler creates function → "go to footer text node, set it to year value"
Runtime:    year changes → run that specific function → update that exact DOM node
            No comparison. No second tree. Direct surgery.
```

Comparison Table

	React (Virtual DOM)	Angular (Incremental DOM)
Memory usage	Higher (stores a second JS UI tree)	Lower (no second tree stored)
CPU usage	More (builds + diffs full trees)	Less (direct instructions)
Speed perception	Very fast for most apps	Very fast, especially on low-end devices
What it compares	Two virtual UI trees	The raw data values (bindings)

> ⚠ **Trap:** Assuming Virtual DOM is always faster than Vanilla JS. Highly optimized Vanilla JS beats both. Virtual DOM lets you write complex apps *without* manually writing painful optimizations.

> **Memory Hook:** React plays "Spot the Difference" with two pictures. Angular uses GPS coordinates to snipe the exact node.

---

4. JSX — JavaScript XML

Core Idea

JSX is a syntax extension that lets you write **HTML-like markup directly inside your JavaScript files**. It looks like HTML, but it's actually JavaScript.

The Problem It Solves

Building UIs with pure JS was verbose and painful:

```
// Pure JavaScript — no JSX (painful)
const el = document.createElement('h1');
el.className = 'greeting';
el.textContent = 'Hello, World!';
```

JSX merges UI structure and logic into one readable place.

JSX is Not HTML — Key Differences

HTML	JSX	Why?

HTML	JSX	Why?
`class="..."`	`className="..."`	`class` is a reserved JS keyword
`for="..."`	`htmlFor="..."`	`for` is a reserved JS keyword
`onclick="..."`	`onClick={...}`	Events use camelCase in JSX
`style="color: red"`	`style={{ color: 'red' }}`	Styles are JS objects in JSX
HTML comments `<!-- -->`	`{/* comment */}`	Comments are JS expressions

JSX in Practice

```
const name = "Angular Dev";
const isLoggedIn = true;

function Greeting() {
  return (
    // Curly braces {} allow any JavaScript expression
    <div className="container">
      <h1>Hello, {name}!</h1>

      {/* Conditional rendering — no *ngIf needed, just JS ternary */}
      {isLoggedIn ? <p>Welcome back!</p> : <p>Please log in.</p>}

      {/* List rendering — no *ngFor, just .map() */}
      <ul>
        {['React', 'Node', 'Angular'].map(tech => (
          <li key={tech}>{tech}</li>
        ))}
      </ul>
    </div>
  );
}
```

Angular vs React — Templating

Task	Angular	React (JSX)
Conditional render	`*ngIf="isVisible"`	`{isVisible ? <A /> : <B />}`
Loop/list	`*ngFor="let x of items"`	`{items.map(x => <li>{x}</li>)}`
Event binding	`(click)="onClick()"`	`onClick={handleClick}`
Data binding	`{{ value }}`	`{value}`
Template file	Separate `.html` file	Same `.jsx` file

> ⚠ **Trap:** Writing `class` instead of `className`. JSX compiles to JavaScript, where `class` is reserved.

> **Memory Hook:** HTML and JavaScript had a baby — that baby is JSX.

---

5. Components — The Building Blocks

Core Idea

A component is a **reusable, self-contained piece of UI**. In modern React, it's simply a **JavaScript function that returns JSX**. Think of it as creating your own custom HTML tag.

Rules for Components

1. **Must be capitalized** — `function Button()` not `function button()`. Lowercase = React treats it as a native HTML tag.

2. **Must return JSX** (or `null` to render nothing).
3. **One top-level element** in the return (use a Fragment `<>` if needed).

Component Example

```
// A simple, reusable Button component
function SubmitButton({ label, onClick }) {
  return (
    <button className="btn-primary" onClick={onClick}>
      {label}
    </button>
  );
}

// Usage — it becomes a custom HTML tag!
export default function App() {
  return (
    <div>
      <SubmitButton label="Save"   onClick={() => console.log('saved')} />
      <SubmitButton label="Cancel" onClick={() => console.log('cancelled')} />
    </div>
  );
}
```

Angular vs React — Components

	Angular	React
Structure	Class + `@Component` decorator	Plain JavaScript function
Template	Separate `.html` file linked via `templateUrl`	JSX inside the same `.jsx` file
Styles	Separate `.css` linked via `styleUrls`	Imported CSS or inline
Selector	`selector: 'app-btn'` → used as `<app-btn>`	Function name → used as `<Button />`
Registration	Must declare in a module or use `standalone`	Just export and import directly

> 💡 **Tip:** The `rfce` shortcut in the **ES7+ React/Redux/React-Native snippets** VS Code extension auto-generates this boilerplate:
>
> ```
> import React from 'react'
>
> function ComponentName() {
>  return ComponentName
> }
>
> export default ComponentName
> ```
>
> Type `rfce` + `Tab` inside any `.jsx` file.

> ⚠ **Trap:** Lowercase component name. `function button()` → React thinks `<button>` is just a native HTML button. Always `PascalCase`.

> 🧠 **Memory Hook:** A component is a custom HTML tag you created yourself.

---

6. Props & Prop Drilling

Core Idea

Props (properties) are the way a **parent component passes data down to a child component**. They are read-only — the child can only read them, never modify them.

Prop Drilling is when you have to pass a prop through multiple layers of components that don't need it, just to get it to a deeply nested child that does.

Props in Practice

```
// Child: receives data as props (destructured from the props object)
function UserCard({ name, role, avatar }) {
  return (
    <div className="card">
      <img src={avatar} alt={name} />
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

// Parent: passes data down using attribute-like syntax
export default function App() {
  return (
    <UserCard
      name="Sarah"
      role="Frontend Engineer"
      avatar="https://example.com/sarah.jpg"
    />
  );
}
```

The Prop Drilling Problem

```
App (has: user data)
 └── Layout
      └── Sidebar
           └── UserProfile ← NEEDS the user data
```

To get `user` from `App` to `UserProfile`, you must pass it through `Layout` and `Sidebar` — even though they don't use it. This is **prop drilling** and it becomes painful at scale.

Solutions to prop drilling:

- **Context API** — React's built-in global state (covered in advanced sessions)
- **State management libraries** — Zustand, Redux Toolkit

Angular vs React — Passing Data

	Angular	React
Parent → Child	`@Input()` decorator on class property	Props object passed as function argument
Child → Parent	`@Output()` + `EventEmitter`	Parent passes a **function as a prop**; child calls it
Mutability	Input properties are read-only by convention	Props are strictly read-only (throws error if mutated)

Child-to-Parent Communication in React

```
// Child calls a function the parent provided
function ChildButton({ onSave }) {
  return <button onClick={onSave}>Save Changes</button>;
}

// Parent provides the function as a prop
export default function Parent() {
  const handleSave = () => console.log("Parent received save event!");

  return <ChildButton onSave={handleSave} />;
}
```

> ⚠ **Trap:** Trying to modify a prop inside the child (`props.name = "John"`). This throws an error. Props flow in one direction only — down.

> **Memory Hook:** Props = component parameters. Prop Drilling = a bucket brigade passing water down a long line.

---

7. Hooks — Superpowers for Functions

Core Idea

Hooks are **special functions (starting with `use`)** that let functional components access React features like **state**, **lifecycle events**, and **context** — things that previously required class components.

Why Hooks Exist

Before Hooks (React < 16.8), to have state or lifecycle methods, you needed:

```
// OLD: Bulky class component
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  componentDidMount()  { console.log("mounted"); }
  componentDidUpdate() { console.log("updated"); }
  componentWillUnmount() { console.log("unmounting"); }

  render() {
    return <button onClick={() => this.setState({ count: this.state.count + 1 })}>
      {this.state.count}
    </button>;
  }
}
```

After Hooks:

```
// NEW: Clean functional component with hooks
function Counter() {
  const [count, setCount] = useState(0);
  useEffect(() => { console.log("mounted/updated"); }, [count]);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

The Most Important Hooks

`useState` — Local State

```
const [value, setValue] = useState(initialValue);
// value    → current state (read-only variable)
// setValue → function to update state and trigger re-render
// useState(initialValue) → starting value

// Example:
const [count, setCount] = useState(0);
const [name,  setName]  = useState("Sarah");
const [items, setItems] = useState([]);

// Updating state:
setCount(count + 1);      // ✅ Correct — triggers re-render
count = count + 1;        // ❌ Wrong — mutates directly, no re-render
```

`useEffect` — Lifecycle & Side Effects

```
// Runs AFTER component mounts (like ngOnInit) — empty dependency array
useEffect(() => {
  fetchUserData();
}, []);

// Runs when `userId` changes (like ngOnChanges)
useEffect(() => {
  fetchUserById(userId);
}, [userId]);

// Cleanup on unmount (like ngOnDestroy)
useEffect(() => {
  const timer = setInterval(() => {}, 1000);
  return () => clearInterval(timer); // Cleanup function
}, []);
```

Angular vs React — Lifecycle

Angular	React Hook	Behavior
`ngOnInit`	`useEffect(() => {}, [])`	Runs once after first render
`ngOnChanges`	`useEffect(() => {}, [dep])`	Runs when `dep` value changes
`ngOnDestroy`	`useEffect` return function	Runs on component unmount
`ngAfterViewInit`	`useEffect(() => {}, [])`	Both run after DOM is ready

Hook Rules (Critical!)

1. **Only call hooks at the top level** — never inside `if`, loops, or nested functions.
2. **Only call hooks inside React functions** — not in regular JS utility functions.

```
// ❌ WRONG — conditional hook
if (isLoggedIn) {
  const [user, setUser] = useState(null); // Breaks hook order!
}

// ✅ CORRECT
const [user, setUser] = useState(null);
if (isLoggedIn) { /* use user here */ }
```

> ⚠ **Trap:** Calling `console.log(count)` immediately after `setCount(count + 1)`. State updates are **asynchronous and batched** — `count` still holds the old value until the next render.

> 🧠 **Memory Hook:** Hooks give "superpowers" (state/lifecycle) to plain JavaScript functions.

---

8. Angular vs React — Full Comparison Map

The Big Picture

	Angular	React
Type	Full **Framework** (the entire kitchen)	**UI Library** (just the oven)
Batteries	Included (router, HTTP, forms, DI, animations)	Not included (bring your own libraries)
Opinionated?	Yes — enforces strict structure	No — you decide the architecture
Learning curve	Steep (TypeScript, decorators, DI, modules)	Moderate (JSX, hooks)

Core Mechanics

Concept	Angular	React
DOM Strategy	Incremental DOM (direct Real DOM updates)	Virtual DOM (memory diff then Real DOM patch)
Data Binding	Two-way `[(ngModel)]`	One-way (data down, events up)
State Updates	Mutable — `this.name = 'x'`; Zone.js detects it	Immutable — must call `setName('x')` explicitly
Templating	HTML with directives (`*ngIf`, `*ngFor`)	JSX (pure JavaScript — `?.`, `.map()`, ternary)
File Structure	Separated (`.html`, `.ts`, `.css` per component)	Co-located (logic + UI + optional CSS in `.jsx`)
Dependency Injection	Built-in (services via constructor injection)	Manual (Props, Context API, or Zustand/Redux)
Component type	Class with `@Component` decorator	Plain function returning JSX
CLI	`ng new`, `ng generate component`	`npm create vite` (then manual structure)

The 3 Mindset Shifts (Angular → React)

1. From Magic to Explicit

```
Angular: this.count++         → Zone.js detects → DOM updates automatically
React:   setCount(count + 1)  → You explicitly tell React to re-render
```

2. From Directives to Pure JavaScript

```
// Angular: *ngIf, *ngFor
// React: Just use JavaScript
{isVisible && <Component />}               // ngIf equivalent
{items.map(item => <li key={item.id}>{item.name}</li>)} // ngFor equivalent
```

3. From Two-Way to One-Way Data Flow

```
Angular: [(ngModel)] → both view and model update each other
React:   Value → from state (down). Changes → via event handlers (up)
```

---

9. React & Node.js — The Relationship

Core Idea

React lives in the **browser** (the dining room). Node.js lives on the **server** (the kitchen). They are completely separate environments that communicate via **HTTP APIs** (the waiter carrying JSON data between them).

The Architecture

```
┌──────────────────────────────────────────────┐
│                   BROWSER                    │
│  ┌──────────────────────────────────────┐    │
│  │             React App                │    │
│  │  (Displays UI, handles user interactions)  │
│  └──────────────────┬───────────────────┘    │
└─────────────────────┼────────────────────────┘
                      │  HTTP Request (fetch/axios)
                      │  ← JSON Response
┌─────────────────────┼────────────────────────┐
│               SERVER│                        │
│  ┌──────────────────┴───────────────────┐    │
│  │          Node.js + Express API       │    │
│  │  (Business logic, auth, DB queries)  │    │
│  └──────────────────────────────────────┘    │
└──────────────────────────────────────────────┘
```

Code: Full React + Node.js Flow

```
// ─── 1. NODE.JS (Backend — runs on your server) ──────────────────
const express = require('express');
const app = express();

app.get('/api/user', (req, res) => {
  // Responds with JSON data — never HTML
  res.json({ name: "Angular Dev turned React Pro", role: "Frontend" });
});

app.listen(3000);
```

```
// ─── 2. REACT (Frontend — runs in the browser) ──────────────────
import { useState, useEffect } from 'react';

export default function UserProfile() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // Fetches from the Node.js API after component mounts
    fetch('/api/user')
      .then(res => res.json())
      .then(data => setUser(data)); // Triggers re-render with data
  }, []);

  if (!user) return <p>Loading...</p>;
  return <div>Welcome, {user.name} — {user.role}</div>;
}
```

Node.js Role in React Development

Phase	Node.js Role
Development	Powers the local dev server (`npm run dev`), manages packages via `npm`
Build	Runs the build tool (Vite) to compile JSX → plain JS/HTML/CSS
Production	Only needed if your backend API is written in Node.js. The built React app is just static files.

> **Key insight:** Running `npm run build` outputs pure static HTML/JS/CSS. You can host that on **any** static server (Nginx, AWS S3, Netlify) — no Node.js needed in production for the frontend.

> ⚠ **Trap:** Thinking React and Node.js are bundled together in production. They are entirely separate. Deploy them independently.

> **Memory Hook:** React = what the user clicks. Node.js = what does the heavy lifting behind the scenes.

10. Creating a React Project on Linux (VS Code + Vite)

Why Vite (Not Create React App)?

	Create React App (CRA)	Vite
Status	⚠ Officially deprecated	✅ The modern standard
Start speed	Slow (Webpack-based)	Instant (native ES modules)
Hot reload	Slow	Blazing fast (HMR)
Bundle size	Heavy	Lightweight

Step-by-Step Setup

```
# Step 1: Open VS Code terminal (Ctrl + `)

# Step 2: Scaffold a new React project with Vite
npm create vite@latest my-react-app -- --template react

# Step 3: Move into the project folder
cd my-react-app

# Step 4: Install all Node packages (this is separate from scaffolding!)
npm install

# Step 5: Start the development server
npm run dev
# → Ctrl + Click the localhost link in the terminal to open in browser
```

What Vite Generates (Project Structure)

```
my-react-app/
├── public/              ← Static assets (favicon, images)
│   └── vite.svg
├── src/                 ← Your entire React application lives here
│   ├── assets/          ← Images, fonts used in JS
│   ├── App.jsx          ← Root component
│   ├── App.css
│   ├── main.jsx         ← Entry point — mounts App to the DOM
│   └── index.css        ← Global styles
├── index.html           ← The single HTML file (has <div id="root">)
├── package.json         ← Project metadata + scripts + dependencies
└── vite.config.js       ← Vite configuration
```

Angular vs React — Project Creation

	Angular	React (Vite)
Command	`ng new my-app`	`npm create vite@latest my-app -- --template react`
Auto-install deps	Yes	No — you run `npm install` manually
What it includes	Full framework: router, testing, forms	Bare minimum UI library only
Configuration	`angular.json`	`vite.config.js`

> ⚠ **Trap:** Forgetting `npm install` before `npm run dev`. Vite only copies files, it does not install them. Skipping this step gives "missing modules" errors.

> **Memory Hook:** `npm create vite` is React's ultra-lightweight version of `ng new`.

---

11. App.jsx — The Root Component

Core Idea

`App.jsx` is the **root component** — the single top-level component that acts as the master container for your entire UI tree. It is the trunk; every other component is a branch.

How It Connects to the Browser

```
 index.html                main.jsx                App.jsx
__________                __________              ________

<div id="root">  ←──  createRoot().render()  ←──  export default App
  (empty)               (mounts here)               (your UI tree)
```

The Files Working Together

```
<!-- index.html — The one HTML file React ships -->
<!DOCTYPE html>
<html>
  <body>
    <div id="root"></div>  <!-- React fills this -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

```
// main.jsx — The entry point. Boots React into the HTML.
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css'; // Global styles imported here

const domNode = document.getElementById('root');
createRoot(domNode).render(<App />);
```

```
// App.jsx — The root component. Organizes your UI tree.
import Header from './components/Header/Header';
import MainContent from './components/MainContent/MainContent';

export default function App() {
  return (
    // Must return a SINGLE top-level element (or Fragment)
    <div className="app-container">
      <Header />
      <MainContent />
    </div>
  );
}
```

Where to Put Global Providers

In Angular, you provide global services in `app.module.ts` or `app.config.ts`. In React, you wrap the `<App />` component in `main.jsx`:

```
// main.jsx — Wrap App with global providers
import { BrowserRouter }    from 'react-router-dom'; // Routing
import { ThemeProvider }    from './context/ThemeContext'; // Custom context

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </BrowserRouter>
);
```

Angular vs React — Root Bootstrap

	Angular	React
Root component	`app.component.ts`	`App.jsx`
Entry point	`main.ts`	`main.jsx`
HTML hook	`<app-root>` tag in `index.html`	`<div id="root">` in `index.html`
Bootstrap call	`bootstrapApplication(AppComponent)`	`createRoot(domNode).render(<App />)`

> ⚠ **Trap:** Returning multiple sibling elements from `App.jsx` without a parent wrapper. JSX must return exactly one top-level element. Use a Fragment if needed.

> **Memory Hook:** App.jsx is the trunk of the tree; all other components are just branches.

---

12. Fragment — The Invisible Wrapper

Core Idea

A Fragment lets you **group multiple elements without adding extra nodes to the DOM**. It renders invisibly — no extra `<div>` wrappers polluting your HTML.

The Problem Without Fragments

```
// ❌ PROBLEM: Component must return ONE element
export default function TableRow() {
  return (
    <div>               {/* This breaks the <table> structure! */}
      <td>Name</td>
      <td>Age</td>
    </div>
  );
}
```

Solution: Fragment

```
// ✅ SOLUTION 1: Shorthand Fragment <> </>
export default function TableRow() {
  return (
    <>
      <td>Name</td>
      <td>Age</td>
    </>
  );
}

// ✅ SOLUTION 2: Full Fragment (required when you need a key prop)
import { Fragment } from 'react';

export default function ItemList({ items }) {
  return items.map(item => (
    <Fragment key={item.id}>   {/* key works here, not on <> */}
      <dt>{item.name}</dt>
      <dd>{item.description}</dd>
    </Fragment>
  ));
}
```

When to Use Full `<React.Fragment>` vs `<>`

Situation	Use
Simple grouping	`<>...</>` (shorthand)
Needs a `key` prop (in `.map()`)	`<Fragment key={id}>...</Fragment>`

Angular vs React — Invisible Wrappers

Angular	React
`<ng-container>`	`<React.Fragment>` or `<>`
Groups elements without adding DOM nodes	Same behavior

> ⚠ **Trap:** `return <h1>Hi</h1> <p>There</p>` — this instantly crashes with "JSX expressions must have one parent element."

> 🧠 **Memory Hook:** Fragment = an invisible ghost wrapper.

---

13. Folder Structure

Core Idea

React has **no enforced folder structure**. Vite gives you an empty `src/`. You design the architecture. Here's the industry-standard pattern for a growing project.

Recommended Folder Structure

```
src/
├── assets/              ← Images, fonts, SVGs
├── components/          ← Globally reusable UI components
│   ├── Button/
│   │   ├── Button.jsx
│   │   └── Button.module.css
│   ├── Navbar/
│   │   └── Navbar.jsx
│   └── Modal/
│       └── Modal.jsx
├── pages/               ← One folder per route/page
│   ├── Home/
│   │   ├── Home.jsx
│   │   └── components/  ← Components used ONLY on this page
│   │       └── HeroBanner.jsx
│   ├── About/
│   │   └── About.jsx
│   └── Dashboard/
│       └── Dashboard.jsx
├── hooks/               ← Custom hooks (useAuth, useFetch, etc.)
│   └── useFetch.js
├── context/             ← React Context providers (global state)
│   └── AuthContext.jsx
├── services/            ← API call functions (axios wrappers, etc.)
│   └── userService.js
├── utils/               ← Pure helper functions (formatDate, etc.)
│   └── helpers.js
├── App.jsx
└── main.jsx
```

Terminal Commands to Set This Up

```
# Create the full structure in one shot
mkdir -p src/components src/pages src/hooks src/context src/services src/utils

# Create a component with its own folder
mkdir -p src/components/Navbar
touch src/components/Navbar/Navbar.jsx
touch src/components/Navbar/Navbar.module.css

# Create a page
mkdir -p src/pages/Home/components
touch src/pages/Home/Home.jsx
touch src/pages/Home/components/HeroBanner.jsx
```

Angular vs React — Folder/Architecture

	Angular	React
Structure enforced?	Yes — CLI creates `component/module/service` folders	No — totally up to you
Component creation	`ng generate component nav` (auto-creates + wires up)	Manually create folder + file
Module system	Modules group components	No modules — just file imports

> ⚠ **Trap:** Over-engineering your folder structure from day one. Start flat and introduce nesting only when a folder becomes hard to navigate.

> **Memory Hook:** Angular provides a pre-built house. React gives you bricks and tells you to draw your own floorplan.

---

14. export default — Making Components Public

Core Idea

By default, everything in a JavaScript file is **private** to that file. `export default` makes a component **public** so other files can import and use it.

The ES6 Module System in React

```
// ─── App.jsx ──────────────────────────────────────────────────────────
// export default makes this function importable from other files
export default function App() {
  return <h1>Hello World</h1>; // return sends JSX back to the caller
}

// ─── main.jsx (Entry Point) ───────────────────────────────────────────
import { createRoot } from 'react-dom/client';
import App from './App';  // Importing the default export — no curly braces

createRoot(document.getElementById('root')).render(<App />);
// The return value of App() (the JSX) gets rendered into the DOM
```

Default Export vs Named Export

	Default Export	Named Export
Syntax	`export default function App()`	`export function App()`
Import syntax	`import App from './App'`	`import { App } from './App'`
How many per file	Only **one**	As many as you want
Name on import	Can rename freely: `import MyApp from './App'`	Must match exactly: `{ App }`
Use case	Main component of a file	Utility functions, constants

Practical Example with Both

```
// utils.js — Multiple named exports
export function formatDate(date) { return date.toLocaleDateString(); }
export const API_URL = 'https://api.example.com';

// UserCard.jsx — One default export + named exports
export const cardStyles = { padding: '16px' }; // named

export default function UserCard({ name }) { // default
  return <div style={cardStyles}>{name}</div>;
}

// App.jsx — Importing both
import UserCard, { cardStyles }    from './UserCard'; // default + named
import { formatDate, API_URL }     from './utils';    // named only
```

Angular vs React — Module System

	Angular	React
Sharing components	Declare in `@NgModule` + export	Just `export default` and `import`
Standard	Angular's module system	Standard ES6 JavaScript modules

> ⚠ **Trap:** Forgetting `export default`. If you import a file that doesn't export anything, React throws: *"Element type is invalid: expected a string or a*

> *class/function but got: undefined."*

> 🧠 **Memory Hook:** `return` builds the TV. `export` plugs it into the wall.

---

15. CSS in React — 3 Ways to Style

Option 1: Global CSS Import

```
/* src/index.css */
body { margin: 0; font-family: sans-serif; }
.btn { padding: 8px 16px; }
```

```
// Imported once in main.jsx — affects ENTIRE app
import './index.css';

// Used with a regular string className
<button className="btn">Click</button>
```

Use for: Resets, global fonts, CSS custom properties (variables).

Option 2: CSS Modules (Best Practice for Component Styles)

```
/* Button.module.css */
.myBtn { background: blue; color: white; border-radius: 4px; }
.myBtn:hover { background: darkblue; }
```

```
import styles from './Button.module.css'; // Import as JS object

export default function Button() {
  // className is accessed like a JS property
  return <button className={styles.myBtn}>Click</button>;
}
// Actual rendered HTML: <button class="Button_myBtn_x8z9">
// Build tool renames it automatically — NO class collisions possible!
```

Use for: Component-specific styles that should never leak to other components.

Option 3: Inline Styles (JavaScript Objects)

```
export default function Alert({ type }) {
  const style = {
    backgroundColor: type === 'error' ? 'red' : 'green', // camelCase!
    padding: '12px',
    marginTop: '8px',
    borderRadius: '4px',
  };

  return <div style={style}>Alert message</div>;
}
```

Use for: Truly dynamic styles that depend on runtime values (not recommended as the primary approach).

Why Inline Styles Use `{{ }}` Double Braces

```
<div style={{ color: 'red' }}>
     ↑                    ↑
     |                    |
     |                    └── Inner {}: the JavaScript object
     └── Outer {}: "I'm writing JavaScript here in JSX"
```

Why CSS Property Names Are camelCase in JS

```
// CSS file:      background-color  (hyphen OK in CSS)
// JS object:     backgroundColor   (hyphens illegal in JS identifiers)
```

Comparison Table

Method	Scope	Dynamic	Best For
`import './style.css'`	Global (entire app)	No	Resets, global themes
`import styles from './x.module.css'`	Local (component only)	No	Component styles
`style={{ color: 'red' }}`	Inline (element only)	Yes	Runtime-dynamic styles

Angular vs React — Styling

	Angular	React
Default scope	Scoped (View Encapsulation)	Global (must use CSS Modules for scope)
Scoped styles	`styleUrls: ['./x.css']` (automatic)	`import styles from './x.module.css'` (explicit)
Dynamic classes	`[ngClass]="{'active': isActive}"`	`className={isActive ? 'active' : ''}`

> ⚠ **Trap:** `import './styles.css'` is GLOBAL — it affects every component in the app, not just the one you imported it into.

> **Memory Hook:** In React, you don't `<link>` CSS in HTML — you `import` it straight into JavaScript.

---

16. Bootstrap in React

Two Meanings of "Bootstrap"

1. **React Bootstrap** — The CSS framework (like Angular's `ngx-bootstrap`)
2. **React Bootstrapping** — The process of mounting React to the DOM (`createRoot().render()`)

Bootstrap CSS Framework in React

The standard Bootstrap library uses jQuery to manipulate the DOM. If Bootstrap changes the DOM behind React's back, React's Virtual DOM loses sync — causing bugs.

Solution: Use `react-bootstrap`, which rebuilds every Bootstrap interactive element as a pure React component controlled by React state.

```
# Install both: the React components AND the Bootstrap CSS
npm install react-bootstrap bootstrap
```

```
// main.jsx — Import Bootstrap CSS globally ONCE
import 'bootstrap/dist/css/bootstrap.min.css';
```

```
// YourComponent.jsx
import { Button, Alert, Modal } from 'react-bootstrap';
import { useState } from 'react';

export default function BootstrapDemo() {
  const [showAlert, setShowAlert] = useState(false);

  return (
    <div className="p-4">
      {/* react-bootstrap Button — same API as regular Bootstrap */}
      <Button variant="primary" onClick={() => setShowAlert(!showAlert)}>
        Toggle Alert
      </Button>

      {/* Controlled by React state — no jQuery needed */}
      <Alert show={showAlert} variant="success" className="mt-3">
        React and Bootstrap are now friends!
      </Alert>
    </div>
  );
}
```

React Bootstrapping (Mounting to DOM)

```
// main.jsx — This IS the "bootstrapping" of the React app
import { createRoot } from 'react-dom/client';
import App from './App';

// 1. Find the empty container
const domNode = document.getElementById('root');

// 2. Create a React root (React 18+ required)
const root = createRoot(domNode);

// 3. Render the app
root.render(<App />);
```

Angular vs React — Bootstrap Comparison

	Angular	React
CSS Bootstrap library	`ngx-bootstrap` or `ng-bootstrap`	`react-bootstrap`
App bootstrapping	`bootstrapApplication(AppComponent)`	`createRoot(domNode).render(<App />)`
Why adapt Bootstrap?	Convert Bootstrap JS to Angular Directives	Convert Bootstrap JS to React Components

> ⚠ **Trap 1:** Adding `<script src="bootstrap.bundle.js">` to `index.html`. Standard Bootstrap JS mutates the Real DOM, breaking React's Virtual DOM sync.

> ⚠ **Trap 2:** Installing `react-bootstrap` but forgetting `import 'bootstrap/dist/css/bootstrap.min.css'`. The package provides the logic (bones); Bootstrap provides the CSS (skin). Without CSS, components look like unstyled HTML.

> 🧠 **Memory Hook:** React-Bootstrap = Bootstrap styles + React brains (no native Bootstrap JS).

---

17. Tailwind CSS in React

Core Idea

Tailwind is a **utility-first CSS framework** — you style directly in JSX using predefined class names. No separate CSS files needed for most styling.

Installation

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

```
// tailwind.config.js — Tell Tailwind where to scan for class names
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: { extend: {} },
  plugins: [],
}
```

```
/* src/index.css — Import Tailwind's base styles */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Using Tailwind in JSX

```
export default function UserCard() {
  return (
    // All styling done with utility classes directly in JSX
    <div className="p-6 max-w-sm mx-auto bg-white rounded-xl shadow-lg flex items-center gap-4">
      <img className="w-12 h-12 rounded-full" src="/avatar.jpg" alt="User" />
      <div>
        <h1 className="text-xl font-semibold text-gray-900">Angular Dev</h1>
        <p className="text-sm text-gray-500">Learning Tailwind in React!</p>
      </div>
    </div>
  );
}
```

Dynamic Classes with Tailwind

```
// ✅ CORRECT — Use full class names in ternary
<button className={hasError ? 'bg-red-500' : 'bg-blue-500'}>
  Submit
</button>

// ❌ WRONG — String concatenation breaks Tailwind
<button className={"bg-" + color + "-500"}>
  Submit
</button>
```

Why? Tailwind scans your files at **build time** for complete class names and generates only the CSS it finds. `"bg-red-500"` gets found. `"bg-" + "red" + "-500"` does not.

Angular vs React — Styling Approaches

	Angular	React + Tailwind
Style method	`styleUrls` (separate CSS file)	Utility classes directly in JSX
Dynamic classes	`[ngClass]="{'active': isActive}"`	`className={isActive ? 'active' : ''}`
Scope	View Encapsulation (auto)	Scoped naturally by component structure

> 🧠 **Memory Hook:** Tailwind = styling with Lego bricks directly inside your HTML.

18. React Execution Flow — From URL to DOM

The Complete Flow (Initial Load → User Interaction)

```
STEP 0: USER TYPES THE URL & HITS ENTER
  └── Browser requests the server
  └── Server sends back one file: index.html

STEP 1: index.html LOADS
  └── Browser parses HTML
  └── Finds <div id="root"> (empty)
  └── Loads the JS bundle (your React app)

STEP 2: main.jsx EXECUTES
  └── createRoot(document.getElementById('root'))
  └── .render(<App />)
  └── React begins executing the App() function

STEP 3: RENDER PHASE (React executes your component functions)
  └── App() runs top-to-bottom
  └── JSX is returned → React builds the Virtual DOM tree
  └── Child components are called and return their JSX
  └── Full Virtual DOM snapshot is created

STEP 4: COMMIT PHASE (React writes to the Real DOM)
  └── React compares new Virtual DOM to previous (diffing)
  └── On first load: everything is "new" → full DOM paint
  └── Browser displays the UI to the user

STEP 5: useEffect RUNS (after DOM is painted)
  └── Effects with [] run once → API calls, subscriptions
  └── If state is updated inside useEffect → triggers re-render

─── USER IS NOW LOOKING AT THE PAGE ──────────────────────────────────

STEP 6: EVENT FIRES (e.g., user clicks a button)
  └── Event handler executes
  └── setState / state setter is called (e.g., setCount(count + 1))

STEP 7: RE-RENDER TRIGGERED
  └── React queues a state update (asynchronous + batched)
  └── Component function re-executes completely from top to bottom

STEP 8: NEW VIRTUAL DOM BUILT
  └── New JSX returned from component function
  └── New Virtual DOM tree created in memory

STEP 9: DIFFING (Virtual DOM Comparison)
  └── React compares new Virtual DOM to old Virtual DOM
  └── Identifies only the changed nodes

STEP 10: COMMIT PHASE (DOM Patch)
  └── Only the changed nodes are updated in the Real DOM
  └── Browser repaints only what changed → lightning fast
  └── useEffect with relevant deps runs again
```

Code Tracing the Flow

```
import { useState, useEffect } from 'react';

export default function ExecutionFlow() {
  // STEP 3: This line runs during Render Phase
  console.log("□ Component rendering...");

  const [count, setCount] = useState(0);

  // STEP 5: This runs AFTER the DOM is painted
  useEffect(() => {
    console.log("□ DOM painted. Count is:", count);
  }, [count]); // Re-runs when count changes (STEP 10 → STEP 5)

  // STEP 6: User click triggers a state update
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked: {count}
    </button>
  );
}
```

Console output on first load:

```
□ Component rendering...
□ DOM painted. Count is: 0
```

After button click:

```
□ Component rendering...
□ DOM painted. Count is: 1
```

Critical Nuance: State Updates Are Async

```
const [count, setCount] = useState(0);

const handleClick = () => {
  setCount(count + 1);
  console.log(count); // ⚠ Still prints 0! (old value)
                      // count updates only on the NEXT render
};
```

Angular vs React — Execution Flow

Phase	Angular	React
Initial load	Router activates → Component Class instantiates → `ngOnInit` → template rendered	`createRoot().render()` → function executes → JSX returned → DOM painted → `useEffect` runs
State change	`this.prop = x` → Zone.js detects async event → Change Detection runs → DOM updated	`setState(x)` → React queues update → function re-runs → Virtual DOM diffed → DOM patched
Idle state	Zone.js monitoring async tasks	Waiting — React does nothing until a state setter is called

> ⚠ **Trap:** Putting heavy calculations or API calls in the component function body. Because the function re-runs on every state change, you'll execute expensive code repeatedly. Always wrap in `useEffect`.

> □ **Memory Hook:** Trigger the spark → Render the blueprint → Commit the build.

---

19. Component Invocation — `<Header />` vs `{Header()}`

Core Idea

Always use JSX tags `<Header />` to render components. Calling them as plain functions `{Header()}` looks equivalent but breaks React's ability to manage state, lifecycle, and the component tree.

Why This Matters

```
import Header from './Header';

export default function App() {
  return (
    <>
      {/* ✅ CORRECT — React manages Header as an independent instance */}
      <Header />

      {/* ❌ ANTI-PATTERN — Dumps JSX output directly, strips React management */}
      {Header()}
    </>
  );
}
```

What React Does Differently with Each

	`<Header />`	`{Header()}`
React's perspective	Creates a React component element	Just a function call that returns JSX
Hooks	Each hook belongs to `Header`'s scope	Hooks are attributed to the **parent** `App` — breaks rules
Lifecycle	React tracks mount/unmount	No tracking
Optimization	Can use `React.memo` to skip re-renders	Cannot
Error boundaries	Can catch errors in this component	Cannot isolate errors

The Hook Crash Explained

```
// Header.jsx
function Header() {
  const [isOpen, setIsOpen] = useState(false); // This hook belongs to Header
  return <nav>{isOpen ? 'Open' : 'Closed'}</nav>;
}

// App.jsx
function App() {
  // {Header()} — React thinks useState is App's hook
  // App's hook order: [ isOpen ]
  // If App itself has hooks, they clash with Header's hooks
  // Result: Crash! "React Hook useState is called in a function that is neither..."
  return {Header()};
}
```

What JSX `<Header />` Actually Compiles To

```
// What you write:
<Header className="main-nav" />

// What Babel compiles it to:
React.createElement(Header, { className: "main-nav" })
// This tells React: "Here is a component type, manage it for me"
```

The `rfce` Shortcut (ES7+ Extension)

Install the **"ES7+ React/Redux/React-Native snippets"** extension in VS Code.

Snippet	What it generates
`rfce`	React Function Component Export (default)
`rfc`	React Function Component (no export)
`useState`	`const [state, setState] = useState(initialState)`
`useEffect`	Complete `useEffect` skeleton

```
Type: rfce  → Tab
Result:
────────────────────────────────────
import React from 'react'

function ComponentName() {
  return (
    <div>ComponentName</div>
  )
}

export default ComponentName
────────────────────────────────────
```

> ⚠ **Trap:** Using `{Header()}` when `Header` contains hooks. React tracks hooks by call order per component. Calling Header as a function collapses its hooks into the parent, breaking the order guarantee.

> **Memory Hook:** `<Header />` hires React to manage the component. `{Header()}` does the work yourself and breaks the app.

---

20. Quick Reference — Traps & Memory Hooks

Topic	⚠ Trap	Memory Hook
SPA	Using `<a href>` instead of `<Link to>`	One HTML shell, infinite JS views
Virtual DOM	Assuming it's faster than optimized Vanilla JS	"Spot the Difference" picture game
JSX	Writing `class` instead of `className`	HTML and JS had a baby
Components	Lowercase component name (`function button`)	A custom HTML tag you created
Props	Modifying props directly inside child	Props = function parameters
Hooks	Calling hooks inside `if` statements	Hooks always in the same order
State	`console.log(count)` after `setCount` expecting new value	State updates are async — old value until next render
Execution	Heavy code in component body (not in `useEffect`)	Trigger → Render → Commit
CSS Global	`import './style.css'` scopes to only that component	Global CSS is truly global
Tailwind	Dynamic class with string concatenation	Write full class names always
Bootstrap	Adding Bootstrap's JS `<script>` to `index.html`	Bootstrap styles + React brains
Bootstrap	Forgetting `import 'bootstrap/dist/css/...'`	Logic = bones, CSS = skin
`export default`	Forgetting to export → "Element type is invalid" error	`return` builds, `export` ships

Topic	⚠ Trap	🧠 Memory Hook
`<Header />`	Using `{Header()}` when it has hooks	`<Header />` hires React; `{Header()}` fires yourself
Vite setup	Forgetting `npm install` before `npm run dev`	Vite drops files; npm install brings them to life
Fragment	Forgetting the `key` prop needs `<Fragment key={}>` not `<>`	Invisible ghost wrapper

---

21. 🗺 Master Summary Flow

This is the mental model that connects everything you learned in this session. Read it top-to-bottom as the journey of a React application.

```
╔═══════════════════════════════════════════════════════════════╗
║                    THE REACT MENTAL MAP                       ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  1. CONCEPT: WHAT IS REACT?                                   ║
║     └── A UI library (not a framework)                        ║
║     └── Handles ONE thing: rendering & updating the view      ║
║     └── You bring your own: router, HTTP, state management    ║
║                                                               ║
║  2. ARCHITECTURE: HOW THE PIECES FIT                          ║
║     └── SPA: One index.html, JavaScript fakes the rest        ║
║     └── Node.js: Separate backend, communicates via JSON API  ║
║     └── Vite: Build tool that compiles JSX → browser JS       ║
║                                                               ║
║  3. THE DOM STRATEGY                                          ║
║     └── Real DOM: Slow to update broadly                      ║
║     └── Virtual DOM: Fast by comparing JS copies in memory    ║
║     └── vs Angular Incremental DOM: Compiles templates to direct ║
║         DOM instructions (less RAM, no virtual copy)          ║
║                                                               ║
║  4. THE COMPONENT SYSTEM                                      ║
║     └── Components: Functions that return JSX                 ║
║     └── JSX: HTML-like syntax compiled to JavaScript          ║
║     └── Fragment: Invisible wrapper to avoid extra DOM nodes  ║
║     └── Invoke as: <Component />  (never Component())         ║
║                                                               ║
║  5. DATA FLOW                                                 ║
║     └── Props: Data flows DOWN (parent → child). Read-only.   ║
║     └── Events: Actions flow UP (child calls parent's function) ║
║     └── Prop Drilling: Pain of passing props through many layers ║
║     └── Solution: Context API or state management library     ║
║                                                               ║
║  6. STATE & REACTIVITY                                        ║
║     └── useState: Store reactive data inside a component      ║
║     └── React watches NOTHING automatically (unlike Angular)  ║
║     └── You MUST call setState to trigger a re-render         ║
║     └── State updates are asynchronous and batched            ║
║                                                               ║
║  7. LIFECYCLE (useEffect)                                     ║
║     └── []     → ngOnInit (run once on mount)                 ║
║     └── [dep]  → ngOnChanges (run when dep changes)           ║
║     └── return → ngOnDestroy (cleanup on unmount)             ║
║                                                               ║
║  8. STYLING OPTIONS (Pick your approach)                       ║
║     └── Global CSS   → index.css, affects whole app           ║
║     └── CSS Modules  → .module.css, scoped to component       ║
║     └── Inline       → style={{ }}, for dynamic values        ║
║     └── Tailwind     → utility classes in JSX, no CSS files   ║
║     └── Bootstrap    → react-bootstrap package, state-controlled ║
║                                                               ║
║  9. MODULE SYSTEM                                             ║
║     └── export default → one main thing per file              ║
```

```
│     └── export (named) → multiple utilities per file              │
│     └── import         → consume from other files                 │
│                                                                     │
│  10. EXECUTION FLOW                                                 │
│      URL typed                                                      │
│       → index.html loads → main.jsx runs → createRoot().render()   │
│       → App() executes → JSX returned → Virtual DOM built          │
│       → Real DOM painted → useEffect([]) runs                      │
│       → [USER SEES UI]                                              │
│       → User interacts → setState called → Re-render queued        │
│       → Component function re-runs → New Virtual DOM built         │
│       → Diffed against old → Only changes applied to Real DOM      │
│       → useEffect([dep]) runs if dep changed                       │
└─────────────────────────────────────────────────────────────────────┘
```

The Angular-to-React Translation Table (Final Reference)

You know this in Angular...	It's this in React
`@Component` decorator	`export default function MyComponent()`
`component.html` template	JSX returned from function
`styleUrls`	CSS Module import or Tailwind
`*ngIf`	`{condition ? <A /> : <B />}`
`*ngFor`	`{items.map(item => <li key={item.id}>)}`
`@Input()`	Props (function argument)
`@Output()` + `EventEmitter`	Function passed as a prop
`ngOnInit`	`useEffect(() => {}, [])`
`ngOnChanges`	`useEffect(() => {}, [dep])`
`ngOnDestroy`	`useEffect` cleanup return
`[(ngModel)]`	`value={state}` + `onChange={setter}`
`ng-container`	`<>` or `<React.Fragment>`
`app.component.ts`	`App.jsx`
`main.ts` + `bootstrapApplication()`	`main.jsx` + `createRoot().render()`
`ng new my-app`	`npm create vite@latest my-app -- --template react`
`ng generate component`	Manually create folder + `.jsx` file (use `rfce` snippet)
`ng serve`	`npm run dev`
`ng build`	`npm run build`
`ngx-bootstrap`	`react-bootstrap`
`Zone.js` (automatic change detection)	You calling `setState` explicitly

---

> ✍ **Session Notes by:** A React beginner with a strong Angular background
> 🤖 **Mentor:** Gemini (formatted & deepened by Claude)
> 📅 **Last updated:** April 2026

> 🚀 **Next topics to explore:** `useContext`, `useReducer`, React Router deep-dive, custom hooks, `React.memo` and performance optimization